

# PROJECT – HIGH LEVEL DESIGN

## on

# Autonomous AI Research Agent

Course Name: Agentic AI

**Institution Name:** Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

| Sr no | Student Name      | Enrolment Number |
|-------|-------------------|------------------|
| 1     | ANIKET JAT        | EN22CS304010     |
| 2     | HIMANSHU SAHRAWAT | EN22CS304010     |
| 3     | SHRIYANSH GUHA    | EN22CS303054     |
| 4     | ROUNAK PATHEKAR   | EN22CS303046     |
| 5     | YASHASVI BAKORE   | EN22CS3030359    |

*Group Name: Group 03D11*

*Project Number: AAI-29*

*Industry Mentor Name:*

*University Mentor Name: Prof. Nishant Shrivastava*

*Academic Year: 2025-26*

## Table of Contents

1. Introduction.
  - 1.1. Scope of the document.
  - 1.2. Intended Audience
  - 1.3. System overview.
2. System Design.
  - 2.1. Application Design
  - 2.2. Process Flow.
  - 2.3. Information Flow.
  - 2.4. Components Design
  - 2.5. Key Design Considerations
  - 2.6. API Catalogue.
3. Data Design.
  - 3.1. Data Model
  - 3.2. Data Access Mechanism
  - 3.3. Data Retention Policies
  - 3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
  - 7.1. Security Aspects
  - 7.2. Performance Aspects
8. References

## **1. Introduction**

### **1.1 Scope of the Document**

This document provides a high-level overview of the design and architecture of the Autonomous AI Research Agent. It explains how the system processes user queries, interacts with external APIs, generates summaries using AI models, and stores the results. The document also highlights system components, data flow, and design considerations.

### **1.2 Intended Audience**

This document is intended for developers, system designers, evaluators, and recruiters who want to understand the system architecture and functionality. It can also be used for academic submission and technical discussions.

### **1.3 System Overview**

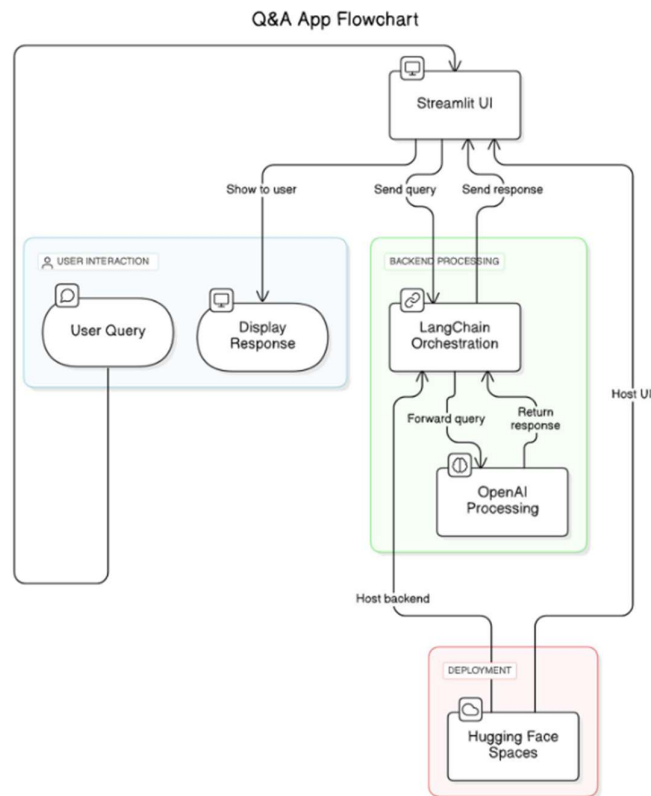
The Autonomous AI Research Agent is an intelligent system that automates the process of research. It accepts a user query, performs real-time web search using the Tavily API, processes the retrieved data, and generates a summarized output using a HuggingFace transformer model.

The system consists of multiple layers including a user interface, backend processing unit, external API integration, and storage. It aims to reduce manual effort in gathering and analyzing information by automating the entire workflow.

## **2. System Design**

### **2.1 Application Design**

The application follows a modular architecture where different components are responsible for specific tasks. The frontend is developed using Streamlit, which provides a simple and interactive interface. The backend is implemented in Python and handles all the logic including API calls and data processing.



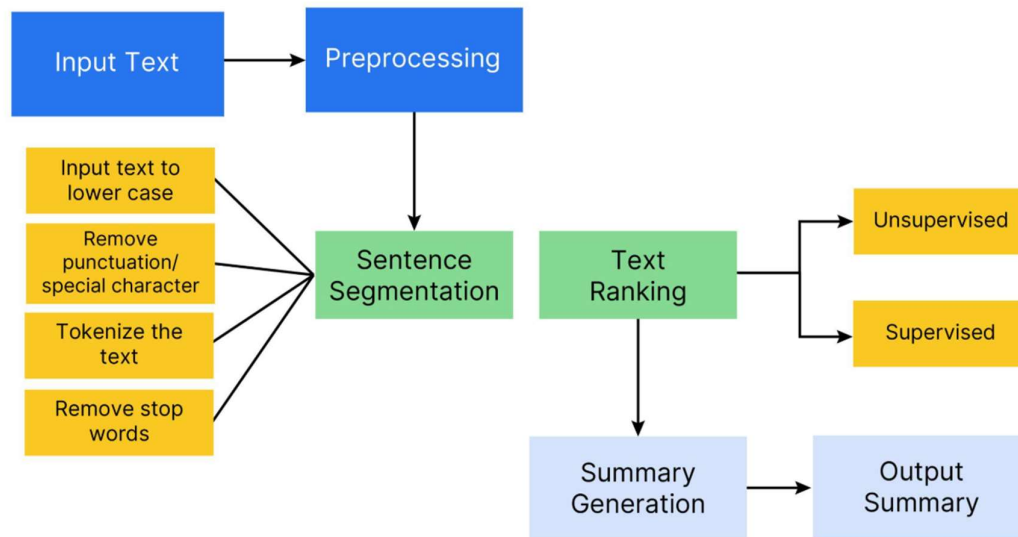
#### Steps:

- User interacts with Streamlit UI
- Backend processes query
- Tavily fetches data
- HuggingFace model generates summary
- Output stored and displayed

The system integrates external services such as Tavily for web search and HuggingFace for AI-based summarization. The output is stored locally, making the system lightweight and easy to deploy.

## 2.2 Process Flow

The system operates in a sequential pipeline. First, the user enters a query through the interface. This query is sent to the backend, which calls the Tavily API to fetch relevant web data. The retrieved data is then processed and passed to the HuggingFace model for summarization. Finally, the summarized output is displayed to the user and stored in a file.



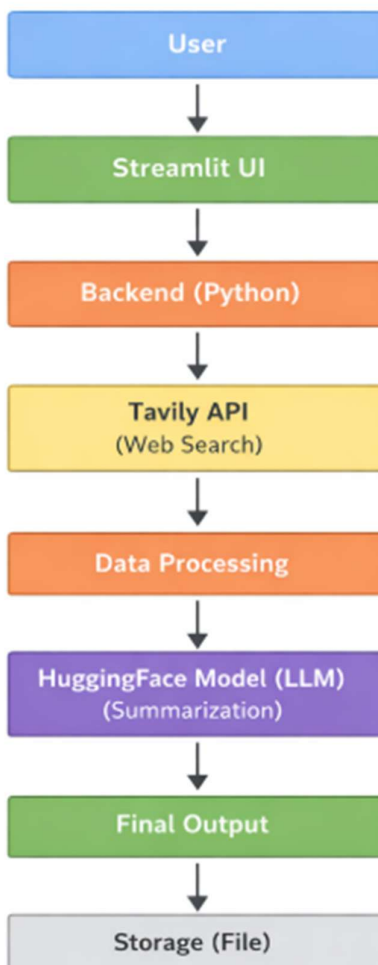
*Input → Search → Process → Summarize → Output*

Steps:

- User inputs query
- Query sent to backend
- Tavily API fetches data
- Data is processed
- AI model generates summary
- Output displayed and saved

### 2.3 Information Flow

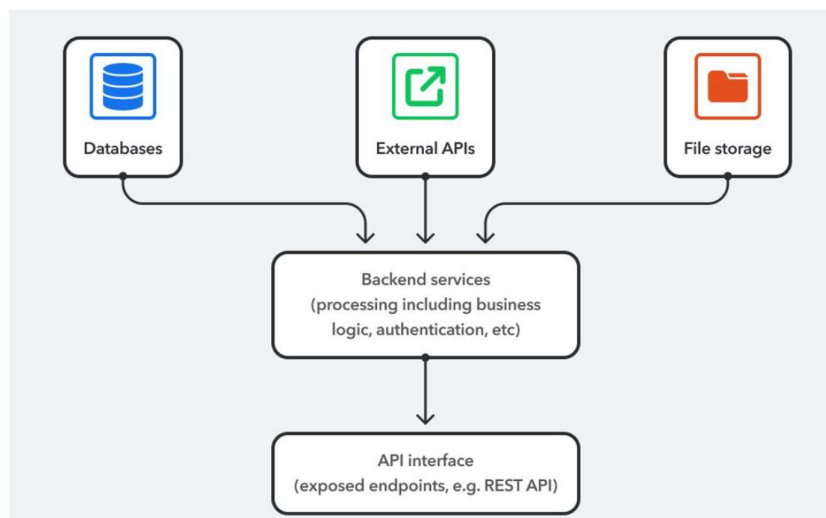
The information flows in a linear manner through the system components. The user interacts with the UI, which sends the request to the backend. The backend communicates with the Tavily API and retrieves data, which is then processed by the AI model. The final result is returned to the UI and stored.



## 2.4 Components Design

The system is divided into multiple components, each responsible for a specific function:

- User Interface (UI): Handles user input and displays output
- Search Component: Interacts with Tavily API to fetch data
- Processing Component: Cleans and organizes the data
- AI Model Component: Generates summarized output
- Storage Component: Saves the results in files



This modular design improves maintainability and allows easy upgrades in the future.

## 2.5 Key Design Considerations

The system is designed keeping scalability, simplicity, and efficiency in mind. A modular approach ensures that components can be updated independently. The use of free APIs reduces cost, while the lightweight design ensures faster performance.

Key Points:

- Modular architecture
- Low cost (free APIs)
- Easy scalability
- Simple deployment
- Efficient processing

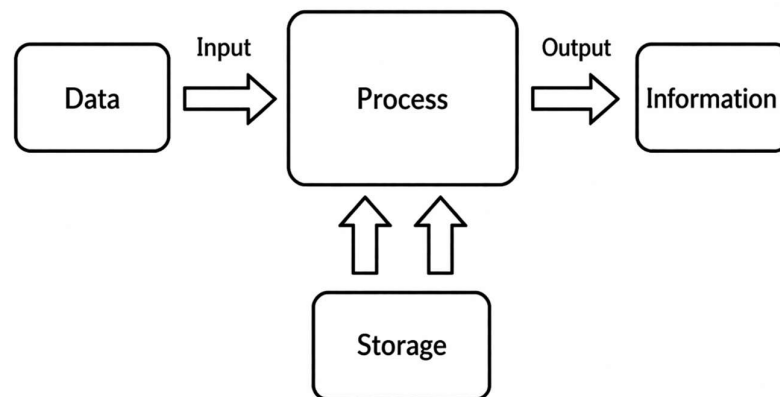
## 2.6 API Catalogue

| API                      | Purpose                |
|--------------------------|------------------------|
| Tavily API               | Real-time web search   |
| HuggingFace Transformers | AI-based summarization |

## 3. Data Design

### 3.1 Data Model

The system handles three types of data: input, intermediate, and output. The input is the user query, the intermediate data is the raw text retrieved from the web, and the output is the summarized result generated by the AI model.



### 3.2 Data Access Mechanism

Data is accessed through API calls and local file operations. The system fetches data using the Tavily API and stores the final output in text files using Python file handling.

### 3.3 Data Retention Policies

The system stores only the final summarized output. No sensitive or personal data is retained. The stored data can be deleted or updated as needed.

### 3.4 Data Migration

Currently, the system uses local storage. In the future, it can be upgraded to use databases or cloud storage solutions for better scalability and accessibility.



## 4. Interfaces

The system provides two types of interfaces:

- Command Line Interface (CLI): For direct execution using terminal
- Web Interface (Streamlit): For user-friendly interaction

## 5. State and Session Management

The system is designed to be stateless. Each request is processed independently. The frontend (Streamlit) manages session-level interactions, while the backend does not maintain persistent state.

## 6. Caching

Currently, caching is not implemented. However, it can be added in the future to improve performance by storing previously fetched results and reducing API calls.

## 7. Non-Functional Requirements

### 7.1 Security Aspects

The system ensures security by storing API keys in environment variables. These keys are not exposed in the frontend. Basic input validation can be implemented to prevent misuse.

### 7.2 Performance Aspects

The system is optimized for fast response times using lightweight AI models. The input size is limited to ensure efficient processing. The architecture supports future enhancements for scalability and performance improvements.

## 8. References

- Tavily API Documentation
- HuggingFace Transformers Documentation
- Streamlit Documentation